Os lab Manual

TASK 1

Practice the following commands in UNIX environment

a) cp b) rm c) mv d) chmod e) ps f) kill

TASK 2

Write a program that makes a copy of a file using standard I/O and system calls

TASK 3

Simulate the following Scheduling algorithms.

a) FCFS b)SJF c)Priority d)Round Robin

TASK 4

Simulate the Producer Consumer problem using semaphores

TASK 5

Simulate the Readers – Writers problem using semaphores.

TASK 6

Simulate the Dining Philosophers problem using semaphores

TASK 7

Simulate Bankers Algorithm for Deadlock Avoidance.

TASK 8

Simulate First Fit and Best Fit algorithms for Memory Management.

TASK 9

Simulate paging technique of memory management.

TASK 10

Simulate page replacement Algorithms. a)FIFO b)LRU

TASK 11

Simulate following Disk Scheduling algorithms.

a) FCFS b) SSTF c) SCAN d) C-SCAN

e) LOOK f) C-LOOK

TASK 12

Simulate file allocation strategies.

a) Sequential b) Indexed c) Linked

Text Books/ References:

1. Operating System Concepts- Abraham Silberchatz , Peter B. Galvin, Greg Gagne 7th Edition,JohnWiley.
2. Operating Systems– Internal and Design Principles Stallings, Fifth Edition–2005, Pearsoneducation/PHI.

TASK 1

Practice the following commands in UNIX environment

a) cp b) rm c) mv d) chmod e) ps f) kill

a) cp

cp stands for **copy**. This command is used to copy files or group of files or directory. It creates an exact image of a file on a disk with different file name. *cp* command require at least two filenames in its arguments.

Syntax:

```
cp [OPTION] Source Destination
cp [OPTION] Source Directory
cp [OPTION] Source-1 Source-2 Source-3 Source-n Directory

First and second syntax is used to copy Source file to Destination
file or Directory.
Third syntax is used to copy multiple Sources(files) to Directory.
```

```
output  :

$ ls
a.txt

$ cp a.txt b.txt

$ ls
a.txt  b.txt
```

b) rm

rm stands for **remove** . rm command is used to remove objects such as files, directories, symbolic links and so on from the file system like UNIX. To be more precise, rm removes references to objects from the filesystem, where those objects might have had multiple references (for example, a file with two different names). **By default, it does not remove directories.** This command normally works silently and you should be very careful while running **rm** command because once you delete the files then you are not able to recover the contents of files and directories.
Syntax:

```
rm [OPTION]... FILE...
```

```
$ ls
a.txt  b.txt  c.txt  d.txt  e.txt

Removing one file at a time
$ rm a.txt

$ ls
b.txt  c.txt  d.txt  e.txt

Removing more than one file at a time
$ rm b.txt c.txt

$ ls
d.txt  e.txt
```

c) mv

mv stands for **move**. mv is used to move one or more files or directories from one place to another in a file system like UNIX. It has two distinct functions:

(i) It renames a file or folder.

(ii) It moves a group of files to a different directory.

No additional space is consumed on a disk during renaming. This command normally **works silently** means no prompt for confirmation.

Syntax:

```
mv [Option] source destination
```

Let us consider 4 files having names **a.txt, b.txt**, and so on till **d.txt**.

To rename the file **a.txt** to **geek.txt(not exist)**:

```
$ ls
a.txt  b.txt  c.txt  d.txt

$ mv a.txt geek.txt

$ ls
```

```
b.txt  c.txt  d.txt  geek.txt
```

d) chmod :

In Unix-like operating systems, the **chmod** command is used to change the access mode of a file.

The name is an abbreviation of **change mode**.

Syntax :

```
chmod [reference][operator][mode] file...
```

The references are used to distinguish the users to whom the permissions apply i.e. they are list of letters that specifies whom to give permissions.

Let's restrict the permission such that the user cannot search the directory EXAM.

```
BEFORE: drwxrwxr-x  mik   mik   EXAM

COMMAND: chmod u=rw EXAM

AFTER: drw-rwxr-x   mik  mik    EXAM
```

e) ps :

Linux provides us a utility called **ps** for viewing information related with the processes on a system which stands as abbreviation for **"Process Status".** ps command is used to list the currently running processes and their PIDs along with some other information depends on different options. It reads the process information from the virtual files in **/proc** file-system. /proc contains virtual files, this is the reason it's referred as a virtual file system.

ps provides numerous options for manipulating the output according to our need.

Syntax –

```
ps [options]
```

```
[root@rhel7 ~]# ps
  PID TTY          TIME CMD
12330 pts/0    00:00:00 bash
21621 pts/0    00:00:00 ps
```

1. Result contains four columns of information.

   Where,

   PID – the unique process ID

   TTY – terminal type that the user is logged into

   TIME – amount of CPU in minutes and seconds that the process has been running

CMD – name of the command that launched the process.

f) kill :

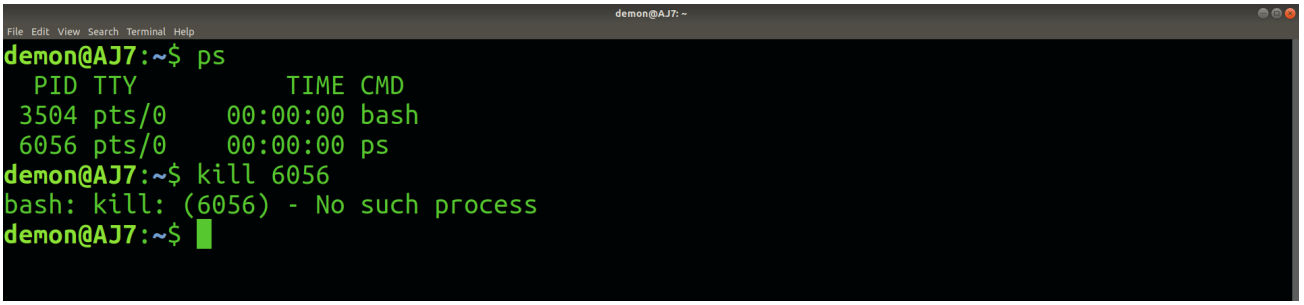
kill command in Linux (located in /bin/kill), is a built-in command which is used to terminate processes manually. *kill* command sends a signal to a process which terminates the process. If the user doesn't specify any signal which is to be sent along with kill command then default *TERM* signal is sent that terminates the process.

Options and examples

1. kill -l : To display all the available signals you can use below command option:

Syntax:

```
$kill -l
```

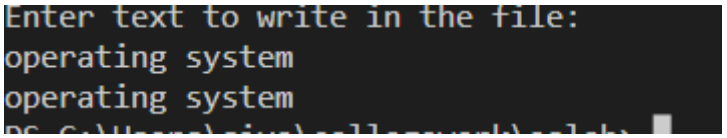


TASK 2

Write a program that makes a copy of a file using standard I/O and system calls

```
#include<stdio.h>
#include<unistd.h>
#include<fcntl.h>
#include<sys/stat.h>
#include<sys/types.h>
int main()
{
        int n,fd;
        char buff[50];
        printf("Enter text to write in the file:\n");
        n=read(0,buff,50);
        fd=open("sample",O_CREAT | O_RDWR,0777);
        write(fd,buff,n);
        write(1,buff,n);
        int close(int fd);
        return 0;
}
```

OUTPUT:



A file with name sample is created .

TASK 3

Simulate the following Scheduling algorithms.

a) FCFS b)SJF c)Priority d)Round Robin

a) FCFS :

```
#include <stdio.h>
// Function to find the waiting time for all processes
int waitingtime(int proc[], int n,
int burst_time[], int wait_time[]) {
  // waiting time for first process is 0
  wait_time[0] = 0;
  // calculating waiting time
  for (int i = 1; i < n ; i++ )
  wait_time[i] = burst_time[i-1] + wait_time[i-1] ;
  return 0;
}
// Function to calculate turn around time
int turnaroundtime( int proc[], int n,
int burst_time[], int wait_time[], int tat[]) {
  // calculating turnaround time by adding
  // burst_time[i] + wait_time[i]
  int i;
  for ( i = 0; i < n ; i++)
  tat[i] = burst_time[i] + wait_time[i];
  return 0;
}
//Function to calculate average time
int avgtime( int proc[], int n, int burst_time[]) {
  int wait_time[n], tat[n], total_wt = 0, total_tat = 0;
  int i;
```

```
  //Function to find waiting time of all processes
  waitingtime(proc, n, burst_time, wait_time);
  //Function to find turn around time for all processes
  turnaroundtime(proc, n, burst_time, wait_time, tat);
  //Display processes along with all details
  printf("Processes  Burst   Waiting Turn around \n");
  // Calculate total waiting time and total turn
  // around time
  for ( i=0; i<n; i++) {
    total_wt = total_wt + wait_time[i];
    total_tat = total_tat + tat[i];
    printf(" %d\t  %d\t\t %d \t%d\n", i+1, burst_time[i], wait_time[i], tat[i]);
  }
  printf("Average waiting time = %f\n", (float)total_wt / (float)n);
  printf("Average turn around time = %f\n", (float)total_tat / (float)n);
  return 0;
}
// main function
int main() {
  //process id's
  int proc[] = { 1, 2, 3};
  int n = sizeof proc / sizeof proc[0];
  //Burst
  int burst_time[] = {5, 8, 12};
  avgtime(proc, n, burst_time);
  return 0;
}
```

OUTPUT:

```
Processes  Burst   Waiting Turn around
 1         5              0      5
 2         8              5      13
 3         12             13     25
Average waiting time = 6.000000
Average turn around time = 14.333333
```

b)SJF :

sjf 1

```
#include<stdio.h>
 int main()
```

```
{
   int bt[20],p[20],wt[20],tat[20],i,j,n,total=0,pos,temp;
   float avg_wt,avg_tat;
   printf("Enter number of process:");
   scanf("%d",&n);

   printf("nEnter Burst Time:n");
   for(i=0;i<n;i++)
   {
      printf("p%d:",i+1);
      scanf("%d",&bt[i]);
      p[i]=i+1;
   }

  //sorting of burst times
   for(i=0;i<n;i++)
   {
      pos=i;
      for(j=i+1;j<n;j++)
      {
         if(bt[j]<bt[pos])
            pos=j;
      }

      temp=bt[i];
      bt[i]=bt[pos];
      bt[pos]=temp;

      temp=p[i];
      p[i]=p[pos];
      p[pos]=temp;
   }

   wt[0]=0;


   for(i=1;i<n;i++)
   {
      wt[i]=0;
```

```
    for(j=0;j<i;j++)
      wt[i]+=bt[j];

    total+=wt[i];
  }

  avg_wt=(float)total/n;
  total=0;

  printf("nProcesst    Burst Time    tWaiting TimetTurnaround Time");
  for(i=0;i<n;i++)
  {
    tat[i]=bt[i]+wt[i];
    total+=tat[i];
    printf("np%dtt  %dtt    %dttt%d",p[i],bt[i],wt[i],tat[i]);
  }

  avg_tat=(float)total/n;
  printf("nnAverage Waiting Time=%f",avg_wt);
  printf("nAverage Turnaround Time=%fn",avg_tat);
}
```

OUTPUT:

```
Enter number of process:2

Enter Burst Time:np1:1
p2:2

Processt    Burst Time    tWaiting TimetTurnaround Time
p1tt  1tt    0ttt1
p2tt  2tt    1ttt3
Average Waiting Time=0.500000
Average Turnaround Time=2.000000n
```

sjf 2

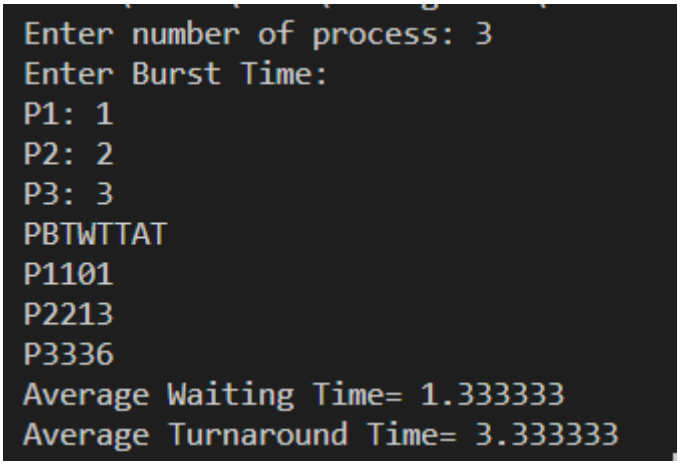
```
#include <stdio.h>
int main()
{
                    int A[100][4]; // Matrix for storing Process Id, Burst
// Time, Average Waiting Time & Average
```

```
// Turn Around Time.
  int i, j, n, total = 0, index, temp;
  float avg_wt, avg_tat;
  printf("Enter number of process: ");
  scanf("%d", &n);
  printf("Enter Burst Time:\n");
  // User Input Burst Time and alloting Process Id.
  for (i = 0; i < n; i++) {
                                printf("P%d: ", i + 1);
                                scanf("%d", &A[i][1]);
                                A[i][0] = i + 1;
  }
  // Sorting process according to their Burst Time.
  for (i = 0; i < n; i++) {
                                index = i;
                                for (j = i + 1; j < n; j++)
                                                if (A[j][1] < A[index][1])
                                                                index = j;
                                                temp = A[i][1];
                                                A[i][1] = A[index][1];
                                                A[index][1] = temp;

                                                temp = A[i][0];
                                                A[i][0] = A[index][0];
                                                A[index][0] = temp;
                                }
                                A[0][2] = 0;
                                // Calculation of Waiting Times
                                for (i = 1; i < n; i++) {
                                                A[i][2] = 0;
                                                for (j = 0; j < i; j++)
                                                A[i][2] += A[j][1];
                                                total += A[i][2];
                                }
                                avg_wt = (float)total / n;
                                total = 0;
                                printf("PBTWTTAT\n");
                                // Calculation of Turn Around Time and printing the
                                // data.
```

```
                for (i = 0; i < n; i++) {
                        A[i][3] = A[i][1] + A[i][2];
                        total += A[i][3];
                        printf("P%d%d%d%d\n", A[i][0],
                        A[i][1], A[i][2], A[i][3]);
                }
                avg_tat = (float)total / n;
                printf("Average Waiting Time= %f", avg_wt);
                printf("\nAverage Turnaround Time= %f", avg_tat);
}
```

OUTPUT:



3

C

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
   int et[20],at[10],n,i,j,temp,st[10],ft[10],wt[10],ta[10];
   int totwt=0,totta=0;
   float awt,ata;
   char pn[10][10],t[10];
   //clrscr();
   printf("Enter the number of process:");
   scanf("%d",&n);
   for(i=0; i<n; i++)
   {
      printf("Enter process name, arrival time& execution time:");
      //flushall();
      scanf("%s%d%d",pn[i],&at[i],&et[i]);
```

```
    }
    for(i=0; i<n; i++)
       for(j=0; j<n; j++)
       {
          if(et[i]<et[j])
          {
             temp=at[i];
             at[i]=at[j];
             at[j]=temp;
             temp=et[i];
             et[i]=et[j];
             et[j]=temp;
             strcpy(t,pn[i]);
             strcpy(pn[i],pn[j]);
             strcpy(pn[j],t);
          }
       }
    for(i=0; i<n; i++)
    {
       if(i==0)
          st[i]=at[i];
       else
          st[i]=ft[i-1];
       wt[i]=st[i]-at[i];
       ft[i]=st[i]+et[i];
       ta[i]=ft[i]-at[i];
       totwt+=wt[i];
       totta+=ta[i];
    }
    awt=(float)totwt/n;
    ata=(float)totta/n;
    printf("\nPname\tarrivaltime\texecutiontime\twaitingtime\ttatime");
    for(i=0; i<n; i++)
       printf("\n%s\t%5d\t\t%5d\t\t%5d\t\t%5d",pn[i],at[i],et[i],wt[i],ta[i]);
    printf("\nAverage waiting time is:%f",awt);
    printf("\nAverage turnaroundtime is:%f",ata);
    getch();
}
```

OUTPUT:

```
Enter the number of process:3
Enter process name, arrival time& execution time:p1 3 5
Enter process name, arrival time& execution time:p2 2 1
Enter process name, arrival time& execution time:p3 1 2

Pname   arrivaltime     executiontime   waitingtime     tatime
p2          2               1               0               1
p3          1               2               2               4
p1          3               5               2               7
Average waiting time is:1.333333
Average turnaroundtime is:4.000000
```

c)Priority :

```
#include<stdio.h>

int main()
{
   int bt[20],p[20],wt[20],tat[20],pr[20],i,j,n,total=0,pos,temp,avg_wt,avg_tat;
   printf("Enter Total Number of Process:");
   scanf("%d",&n);

   printf("\nEnter Burst Time and Priority\n");
   for(i=0;i<n;i++)
   {
      printf("\nP[%d]\n",i+1);
      printf("Burst Time:");
      scanf("%d",&bt[i]);
      printf("Priority:");
      scanf("%d",&pr[i]);
      p[i]=i+1;         //contains process number
   }

   //sorting burst time, priority and process number in ascending order using selection sort
   for(i=0;i<n;i++)
   {
      pos=i;
      for(j=i+1;j<n;j++)
      {
         if(pr[j]<pr[pos])
            pos=j;
      }

      temp=pr[i];
```

```
        pr[i]=pr[pos];
        pr[pos]=temp;

        temp=bt[i];
        bt[i]=bt[pos];
        bt[pos]=temp;

        temp=p[i];
        p[i]=p[pos];
        p[pos]=temp;
    }

    wt[0]=0; //waiting time for first process is zero

    //calculate waiting time
    for(i=1;i<n;i++)
    {
        wt[i]=0;
        for(j=0;j<i;j++)
            wt[i]+=bt[j];

        total+=wt[i];
    }

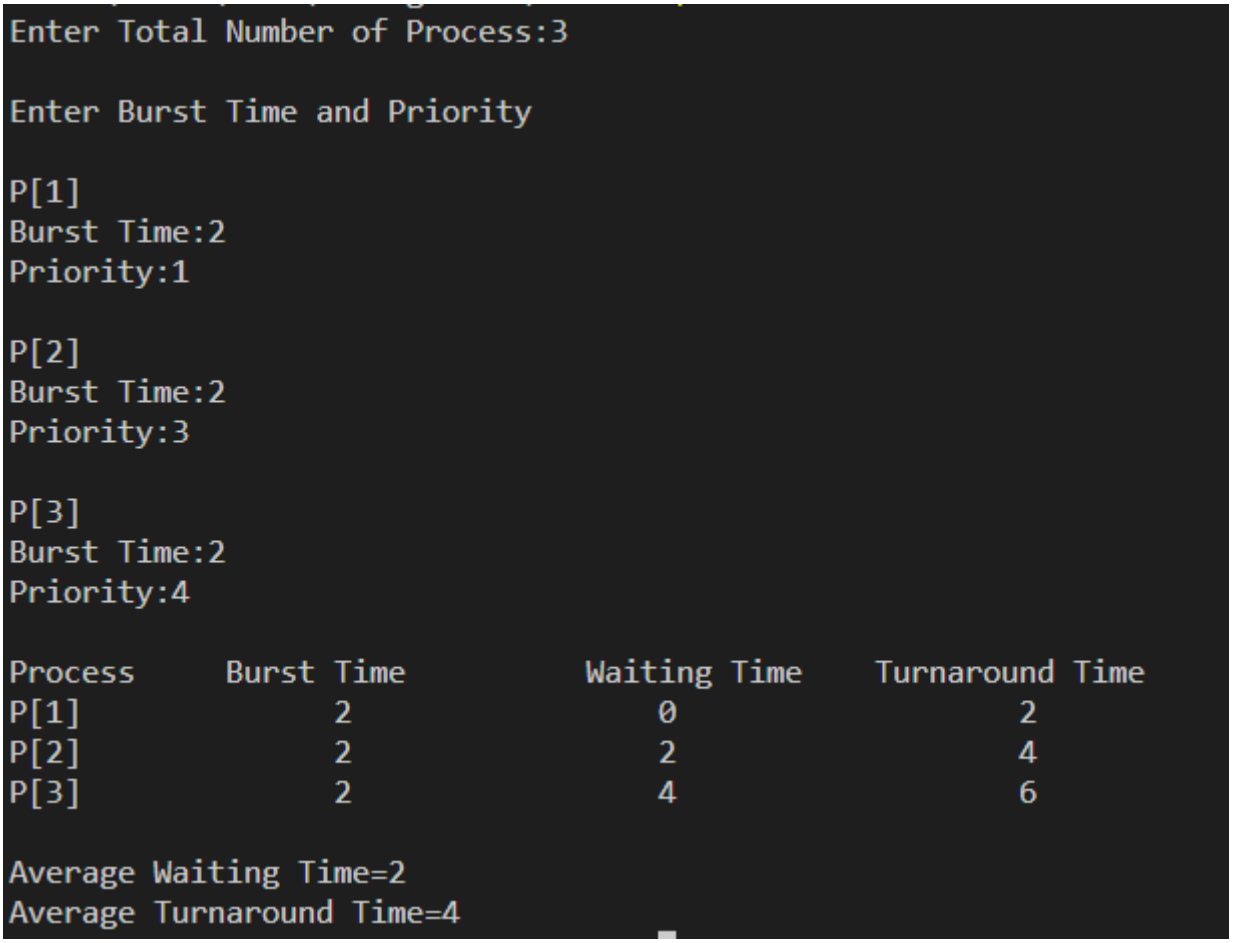
    avg_wt=total/n;      //average waiting time
    total=0;

    printf("\nProcess\t    Burst Time    \tWaiting Time\tTurnaround Time");
    for(i=0;i<n;i++)
    {
        tat[i]=bt[i]+wt[i];     //calculate turnaround time
        total+=tat[i];
        printf("\nP[%d]\t\t  %d\t\t    %d\t\t\t%d",p[i],bt[i],wt[i],tat[i]);
    }

    avg_tat=total/n;     //average turnaround time
    printf("\n\nAverage Waiting Time=%d",avg_wt);
    printf("\nAverage Turnaround Time=%d\n",avg_tat);

  return 0;
  }
```

OUTPUT:



d)Round

```
#include<stdio.h>
#include<conio.h>

void main()
{
   // initlialize the variable name
   int i, NOP, sum=0,count=0, y, quant, wt=0, tat=0, at[10], bt[10], temp[10];
   float avg_wt, avg_tat;
   printf(" Total number of process in the system: ");
   scanf("%d", &NOP);
   y = NOP; // Assign the number of process to variable y

// Use for loop to enter the details of the process like Arrival time and the Burst Time
for(i=0; i<NOP; i++)
{
printf("\n Enter the Arrival and Burst time of the Process[%d]\n", i+1);
```

```
printf(" Arrival time is: \t");  // Accept arrival time
scanf("%d", &at[i]);
printf(" \nBurst time is: \t"); // Accept the Burst time
scanf("%d", &bt[i]);
temp[i] = bt[i]; // store the burst time in temp array
}
// Accept the Time qunat
printf("Enter the Time Quantum for the process: \t");
scanf("%d", &quant);
// Display the process No, burst time, Turn Around Time and the waiting time
printf("\n Process No \t\t Burst Time \t\t TAT \t\t Waiting Time ");
for(sum=0, i = 0; y!=0; )
{
if(temp[i] <= quant && temp[i] > 0) // define the conditions
{
   sum = sum + temp[i];
   temp[i] = 0;
   count=1;
   }
   else if(temp[i] > 0)
   {
      temp[i] = temp[i] - quant;
      sum = sum + quant;
   }
   if(temp[i]==0 && count==1)
   {
      y--; //decrement the process no.
      printf("\nProcess No[%d] \t\t %d\t\t\t\t %d\t\t\t %d", i+1, bt[i],
sum-at[i], sum-at[i]-bt[i]);
      wt = wt+sum-at[i]-bt[i];
      tat = tat+sum-at[i];
      count =0;
   }
   if(i==NOP-1)
   {
      i=0;
   }
   else if(at[i+1]<=sum)
   {
```

```
      i++;
    }
    else
    {
      i=0;
    }
  }
  // represents the average waiting time and Turn Around time
  avg_wt = wt * 1.0/NOP;
  avg_tat = tat * 1.0/NOP;
  printf("\n Average Turn Around Time: \t%f", avg_wt);
  printf("\n Average Waiting Time: \t%f", avg_tat);
  getch();
  }
```

OUTPUT:

```
Total number of process in the system: 2

Enter the Arrival and Burst time of the Process[1]
Arrival time is:       1

Burst time is:  3

Enter the Arrival and Burst time of the Process[2]
Arrival time is:       3

Burst time is:  4
Enter the Time Quantum for the process:         2

Process No              Burst Time              TAT             Waiting Time
Process No[1]           3                               2                       -1
Process No[2]           4                               4                       0
Average Turn Around Time:       -0.500000
Average Waiting Time:   3.000000
```

TASK 4

Simulate the Producer Consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>

int mutex = 1;
int full = 0;
int empty = 10, x = 0;
```

```
void producer()
{
	--mutex;
	++full;
	--empty;
	x++;
	printf("\nProducer produces"
		"item %d",
		x);
	++mutex;
}

void consumer()
{
	--mutex;
	--full;
	++empty;
	printf("\nConsumer consumes "
		"item %d",
		x);
	x--;
	++mutex;
}

int main()
{
	int n, i;
	printf("\n1. Press 1 for Producer"
		"\n2. Press 2 for Consumer"
		"\n3. Press 3 for Exit");

#pragma omp critical

	for (i = 1; i > 0; i++) {

		printf("\nEnter your choice:");
		scanf("%d", &n);

		switch (n) {
		case 1:
```

```
                if ((mutex == 1)
                        && (empty != 0)) {
                        producer();
                }
                else {
                        printf("Buffer is full!");
                }
                break;
        case 2:
                if ((mutex == 1)
                        && (full != 0)) {
                        consumer();
                }
                else {
                        printf("Buffer is empty!");
                }
                break;
        case 3:
                exit(0);
                break;
        }
    }
}
```

OUTPUT:

```
1. Press 1 for Producer
2. Press 2 for Consumer
3. Press 3 for Exit
Enter your choice:1

Producer producesitem 1
Enter your choice:2

Consumer consumes item 1
Enter your choice:2
Buffer is empty!
Enter your choice:1

Producer producesitem 1
Enter your choice:1

Producer producesitem 2
Enter your choice:3
```

TASK 5

Simulate the Readers – Writers problem using semaphores.

```
#include<semaphore.h>
#include<stdio.h>
#include<stdlib.h>
sem_t x,y;
pthread_t tid;
pthread_t writerthreads[100],readerthreads[100];
int readercount;

void *reader(void* param)
{
   sem_wait(&x);
   readercount++;
   if(readercount==1)
   sem_wait(&y);
   sem_post(&x);
   printf("\n%d reader is inside",readercount);
   sem_wait(&x);
   readercount--;
   if(readercount==0)
   {
      sem_post(&y);
   }
   sem_post(&x);
   printf("\n%d Reader is leaving",readercount+1);
}

void *writer(void* param)
{
   printf("\nWriter is trying to enter");
   sem_wait(&y);
   printf("\nWriter has entered");
   sem_post(&y);
   printf("\nWriter is leaving");
}

int main()
{
   int n2,i;
```

```
    printf("Enter the number of readers:");
    scanf("%d",&n2);
    int n1[n2];
    sem_init(&x,0,1);
    sem_init(&y,0,1);
    for(i=0;i<n2;i++)
    {
        pthread_create(&writerthreads[i],NULL,reader,NULL);
        pthread_create(&readerthreads[i],NULL,writer,NULL);
    }
    for(i=0;i<n2;i++)
    {
        pthread_join(writerthreads[i],NULL);
        pthread_join(readerthreads[i],NULL);
    }

}
```

output :

```
Enter the number of readers:3

1 reader is inside
Writer is trying to enter
2 reader is inside
2 Reader is leaving
2 reader is inside
Writer is trying to enter
Writer is trying to enter
2 Reader is leaving
Writer has entered
1 Reader is leaving
Writer has entered
Writer is leaving
Writer has entered
Writer is leaving
Writer is leaving
```

TASK 6

Simulate the Dining Philosophers problem using semaphores

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>
```

```
#include<unistd.h>

sem_t room;
sem_t chopstick[5];

void * philosopher(void *);
void eat(int);
int main()
{
	int i,a[5];
	pthread_t tid[5];

	sem_init(&room,0,4);

	for(i=0;i<5;i++)
		sem_init(&chopstick[i],0,1);

	for(i=0;i<5;i++){
		a[i]=i;
		pthread_create(&tid[i],NULL,philosopher,(void *)&a[i]);
	}
	for(i=0;i<5;i++)
		pthread_join(tid[i],NULL);
}

void * philosopher(void * num)
{
	int phil=*(int *)num;

	sem_wait(&room);
	printf("\nPhilosopher %d has entered room",phil);
	sem_wait(&chopstick[phil]);
	sem_wait(&chopstick[(phil+1)%5]);

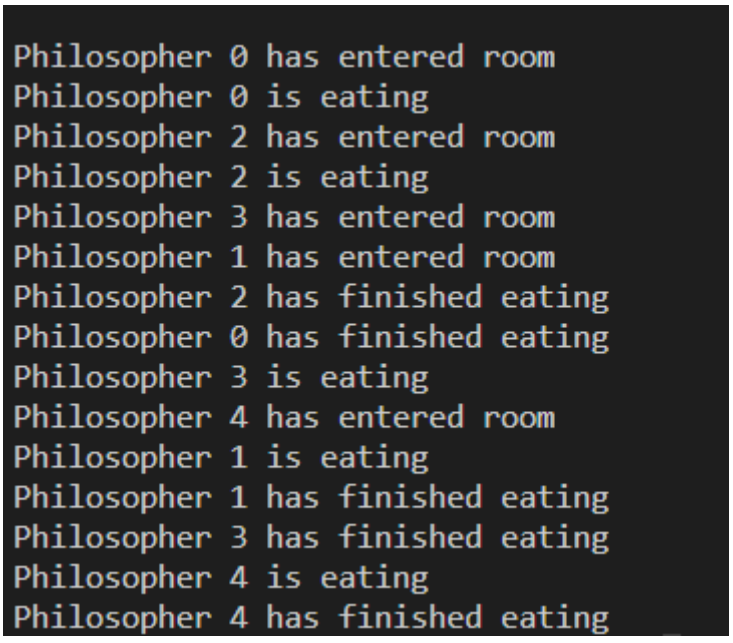
	eat(phil);
```

```
    sleep(2);
    printf("\nPhilosopher %d has finished eating",phil);

    sem_post(&chopstick[(phil+1)%5]);
    sem_post(&chopstick[phil]);
    sem_post(&room);
}

void eat(int phil)
{
    printf("\nPhilosopher %d is eating",phil);
}
```

output :




```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define NUM_PHILOSOPHERS 5
#define NUM_CHOPSTICKS 5

void dine(int n);
```

```
pthread_t philosopher[NUM_PHILOSOPHERS];
pthread_mutex_t chopstick[NUM_CHOPSTICKS];

int main()
{
 // Define counter var i and status_message
 int i, status_message;
 void *msg;

 // Initialise the semaphore array
 for (i = 1; i <= NUM_CHOPSTICKS; i++)
 {
  status_message = pthread_mutex_init(&chopstick[i], NULL);
  // Check if the mutex is initialised successfully
  if (status_message == -1)
  {
   printf("\n Mutex initialization failed");
   exit(1);
  }
 }

 // Run the philosopher Threads using *dine() function
 for (i = 1; i <= NUM_PHILOSOPHERS; i++)
 {
  status_message = pthread_create(&philosopher[i], NULL, (void *)dine, (int
*)i);
  if (status_message != 0)
  {
   printf("\n Thread creation error \n");
   exit(1);
  }
 }

 // Wait for all philosophers threads to complete executing (finish dining)
before closing the program
```

```
 for (i = 1; i <= NUM_PHILOSOPHERS; i++)
 {
  status_message = pthread_join(philosopher[i], &msg);
  if (status_message != 0)
  {
   printf("\n Thread join failed \n");
   exit(1);
  }
 }

 // Destroy the chopstick Mutex array
 for (i = 1; i <= NUM_CHOPSTICKS; i++)
 {
  status_message = pthread_mutex_destroy(&chopstick[i]);
  if (status_message != 0)
  {
   printf("\n Mutex Destroyed \n");
   exit(1);
  }
 }
 return 0;
}
void dine(int n)
{
 printf("\nPhilosopher % d is thinking ", n);

 // Philosopher picks up the left chopstick (wait)
 pthread_mutex_lock(&chopstick[n]);

 // Philosopher picks up the right chopstick (wait)
 pthread_mutex_lock(&chopstick[(n + 1) % NUM_CHOPSTICKS]);

 // After picking up both the chopstick philosopher starts eating
 printf("\nPhilosopher % d is eating ", n);
 sleep(3);
```

```
    // Philosopher places down the left chopstick (signal)
    pthread_mutex_unlock(&chopstick[n]);

    // Philosopher places down the right chopstick (signal)
    pthread_mutex_unlock(&chopstick[(n + 1) % NUM_CHOPSTICKS]);

    // Philosopher finishes eating
    printf("\nPhilosopher % d Finished eating ", n);
  }
```

7.Program to Simulate bankers algorithm for DeadLock Avoidance

```
// Banker's Algorithm
#include <stdio.h>
int main()
{
 // P0, P1, P2, P3, P4 are the Process names here

 int n, m, i, j, k;
 n = 5; // Number of processes
 m = 3; // Number of resources
 int alloc[5][3] = { { 0, 1, 0 }, // P0 // Allocation Matrix
      { 2, 0, 0 }, // P1
      { 3, 0, 2 }, // P2
      { 2, 1, 1 }, // P3
      { 0, 0, 2 } }; // P4

 int max[5][3] = { { 7, 5, 3 }, // P0 // MAX Matrix
     { 3, 2, 2 }, // P1
     { 9, 0, 2 }, // P2
     { 2, 2, 2 }, // P3
     { 4, 3, 3 } }; // P4

 int avail[3] = { 3, 3, 2 }; // Available Resources

 int f[n], ans[n], ind = 0;
 for (k = 0; k < n; k++) {
  f[k] = 0;
 }
 int need[n][m];
 for (i = 0; i < n; i++) {
  for (j = 0; j < m; j++)
   need[i][j] = max[i][j] - alloc[i][j];
 }
 int y = 0;
```

```
for (k = 0; k < 5; k++) {
 for (i = 0; i < n; i++) {
  if (f[i] == 0) {

   int flag = 0;
   for (j = 0; j < m; j++) {
    if (need[i][j] > avail[j]){
     flag = 1;
     break;
    }
   }

   if (flag == 0) {
    ans[ind++] = i;
    for (y = 0; y < m; y++)
     avail[y] += alloc[i][y];
    f[i] = 1;
   }
  }
 }
}

int flag = 1;

for(int i=0;i<n;i++)
{
if(f[i]==0)
{
 flag=0;
 printf("The following system is not safe");
 break;
}
}

if(flag==1)
{
printf("Following is the SAFE Sequence\n");
for (i = 0; i < n - 1; i++)
 printf(" P%d ->", ans[i]);
printf(" P%d", ans[n - 1]);
}


 return (0);

}
```

OUTPUT:

```
Following is the SAFE Sequence
 P1 -> P3 -> P4 -> P0 -> P2
```

8 .Simulate The First Fit and The best fit algorithms for memory management

First Fit Algorithm Program :

```
#include <stdio.h>

void implimentFirstFit(int blockSize[], int blocks, int processSize[], int processes)
{
    // This will store the block id of the allocated block to a process int
    allocate[processes];

    // initially assigning -1 to all allocation indexes

    // means nothing is allocated currently for(int
    i = 0; i < processes; i++)
    {
                allocate[i] = -1;

    }


    // take each process one by one and find

    // first block that can accomodate it for
    (int i = 0; i < processes; i++)
    {
        for (int j = 0; j < blocks; j++) {

            if (blockSize[j] >= processSize[i])

            {// allocate block j to p[i] process

            allocate[i] = j

// Reduce size of block j as it has accomodated p[i] blockSize[j] -= processSize[i];

                break;

            }

        }

    }


    printf("\nProcess No.\tProcess Size\tBlock no.\n"); for (int i =
    0; i < processes; i++)
    {
        printf("%d \t\t\t %d \t\t\t", i+1, processSize[i]); if
```

```
        (allocate[i] != -1)
            printf("%d\n",allocate[i] + 1); else
            printf("Not Allocated\n");
    }
}
void main()
{
    int blockSize[] = {30, 5, 10};
    int processSize[] = {10, 6, 9};
    int m = sizeof(blockSize)/sizeof(blockSize[0]);
    int n = sizeof(processSize)/sizeof(processSize[0]);

    implimentFirstFit(blockSize, m, processSize, n);



    }
```

OUTPUT:

Process No. Process Size Block no.

1 10 1

2 6 1

3 9

Best Fit Algorithm Program :

```
#include <stdio.h>

void implimentBestFit(int blockSize[], int blocks, int processSize[], int processes)
{
    // This will store the block id of the allocated block to a process int
    allocation[processes];

    // initially assigning -1 to all allocation indexes

    // means nothing is allocated currently for(int
    i = 0; i < processes; i++){
        allocation[i] = -1;

    }

    // pick each process and find suitable blocks

    // according to its size ad assign to it for (int
    i=0; i<processes; i++)
    {


        int indexPlaced = -1;

        for (int j=0; j<blocks; j++)

        {

            if (blockSize[j] >= processSize[i])

            {

                // place it at the first block fit to accomodate process if
                (indexPlaced == -1)
                    indexPlaced = j;


                // if any future block is better that is

                // any future block with smaller size encountered

                // that can accomodate the given process else if
                (blockSize[j] < blockSize[indexPlaced])
                    indexPlaced = j;

            }

        }


        // If we were successfully able to find block for the process if
        (indexPlaced != -1)
        {
```

```
            // allocate this block j to process p[i]
            allocation[i] = indexPlaced;

            // Reduce available memory for the block
            blockSize[indexPlaced] -= processSize[i];
        }
    }

    printf("\nProcess No.\tProcess Size\tBlock no.\n"); for (int i =
    0; i < processes; i++)
    {
        printf("%d \t\t\t %d \t\t\t", i+1, processSize[i]); if
        (allocation[i] != -1)
            printf("%d\n",allocation[i] + 1); else
            printf("Not Allocated\n");
    }
}

// Driver code int
main()
{
    int blockSize[] = {50, 20, 100, 90};
    int processSize[] = {10, 30, 60, 30};
    int blocks = sizeof(blockSize)/sizeof(blockSize[0]);
    int processes = sizeof(processSize)/sizeof(processSize[0]);

    implimentBestFit(blockSize, blocks, processSize, processes);

    return 0 ;
}
```

Output:

Process No.	Process Size	Block no.
1	10	2
2	30	1
3	60	4
4	30	4

9 .Simulate Paging Technique of Memory Management

```
#include<stdio.h>
#include<conio.h>
main()
{
 int ms, ps, nop, np, rempages, i, j, x, y, pa, offset;
 int s[10], fno[10][20];

clrscr();

printf("\nEnter the memory size -- ");
scanf("%d",&ms);

printf("\nEnter the page size -- ");
scanf("%d",&ps);

nop = ms/ps;

printf("\nThe no. of pages available in memory are -- %d ",nop);


printf("\nEnter number of processes -- ");
 scanf("%d",&np);
rempages = nop;
for(i=1;i<=np;i++)

{

printf("\nEnter no. of pages required for p[%d]-- ",i);
 scanf("%d",&s[i]);

if(s[i] >rempages)

{


printf("\nMemory is Full");
break;
}

rempages = rempages - s[i];


printf("\nEnter pagetable for p[%d] --- ",i);
```

```
 for(j=0;j<s[i];j++)
scanf("%d",&fno[i][j]);
```

```
printf("\nEnter Logical Address to find Physical Address ");
 printf("\nEnter process no. and pagenumber and offset -- ");

 scanf("%d %d %d",&x,&y, &offset);



 if(x>np || y>=s[i] || offset>=ps)
 printf("\nInvalid Process or Page Number or offset");

 else
 { pa=fno[x][y]*ps+offset;
 printf("\nThe Physical Address is -- %d",pa);

 }
 getch();
 }
```

INPUT :

```
Enter the memory size – 1000
Enter the page size -- 100
The no. of pages available in memory are -- 10
Enter number of processes -- 3
Enter no. of pages required for p[1]-- 4
Enter pagetable for p[1] --- 8 6 9 5
Enter no. of pages required for p[2]-- 5
Enter pagetable for p[2] --- 1 4 5 7 3
Enter no. of pages required for p[3]-- 5
```

OUTPUT:

```
Memory is Full
Enter Logical Address to find Physical Address
```

Enter process no. and pagenumber and offset -- 2 3 60
The Physical Address is -- 760

IMPLEMENTATION OF FIFO PAGE REPLACEMENT ALGORITHM

10 A)AIM

To write a c program to implement FIFO page replacement algorithm

ALGORITHM

1. Start the process
2. Declare the size with respect to page length
3. Check the need of replacement from the page to memory
4. Check the need of replacement from old page to new page in memory
5. Forma queue to hold all pages
6. Insert the page require memory into the queue
7. Check for bad replacement and page fault
8. Get the number of processes to be inserted
9. Display the values
10. Stop the process

PROGRAM:

```
#include<stdio.h>
int main()
{
int i,j,n,a[50],frame[10],no,k,avail,count=0;
        printf("\n ENTER THE NUMBER OF PAGES:\n");
scanf("%d",&n);
        printf("\n ENTER THE PAGE NUMBER :\n");
        for(i=1;i<=n;i++)
        scanf("%d",&a[i]);
        printf("\n ENTER THE NUMBER OF FRAMES :");
        scanf("%d",&no);
```

```
for(i=0;i<no;i++)
        frame[i]= -1;
                j=0;
                printf("\tref string\t page frames\n");
for(i=1;i<=n;i++)
                {
                        printf("%d\t\t",a[i]);
                        avail=0;
                        for(k=0;k<no;k++)
if(frame[k]==a[i])
                                avail=1;
                        if (avail==0)
                        {
                                frame[j]=a[i];
                                j=(j+1)%no;
                                count++;
                                for(k=0;k<no;k++)
                                printf("%d\t",frame[k]);
}
                        printf("\n");
}
                printf("Page Fault Is %d",count);
                return 0;
}
```

OUTPUT:

```
ENTER THE NUMBER OF PAGES:  20
ENTER THE PAGE NUMBER :      7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
ENTER THE NUMBER OF FRAMES :3
    ref string      page frames
7           7     -1     -1
0           7     0      -1
1           7     0      1
2           2     0      1
0
3           2     3      1
0           2     3      0
4           4     3      0
2           4     2      0
3           4     2      3
0           0     2      3
3
2
```

1 0 1 3
2 0 1 2
0
1
7 7 1 2
0 7 0 2
1 7 0 1
Page Fault Is 15

10 B)AIM:

To write a c program to implement LRU page replacement algorithm

ALGORITHM :

1. Start the process

2. Declare the size

3. Get the number of pages to be inserted

4. Get the value

5. Declare counter and stack

6. Select the least recently used page by counter value

7. Stack them according the selection.

8. Display the values

9. Stop the process

PROGRAM:

```
#include<stdio.h>
main()
{
int q[20],p[50],c=0,c1,d,f,i,j,k=0,n,r,t,b[20],c2[20];
printf("Enter no of pages:");
scanf("%d",&n);
printf("Enter the reference string:");
```

```
for(i=0;i<n;i++)
        scanf("%d",&p[i]);
printf("Enter no of frames:");
scanf("%d",&f);
q[k]=p[k];
printf("\n\t%d\n",q[k]);
c++;
k++;
for(i=1;i<n;i++)
        {
                c1=0;
                for(j=0;j<f;j++)
                {
                        if(p[i]!=q[j])
                        c1++;
                }
                if(c1==f)
                {
                        c++;
                        if(k<f)
                        {
                                q[k]=p[i];
                                k++;
                                for(j=0;j<k;j++)
                                printf("\t%d",q[j]);
                                printf("\n");
                        }
                        else
                        {
                                for(r=0;r<f;r++)
                                {
                                        c2[r]=0;
                                        for(j=i-1;j<n;j--)
                                        {
                                        if(q[r]!=p[j])
                                        c2[r]++;
                                        else
                                        break;
                                }
                        }
                        for(r=0;r<f;r++)
                         b[r]=c2[r];
                        for(r=0;r<f;r++)
                        {
                                for(j=r;j<f;j++)
```

```
                        {
                                if(b[r]<b[j])
                                {
                                        t=b[r];
                                        b[r]=b[j];
                                        b[j]=t;
                                }
                        }
                }
                for(r=0;r<f;r++)
                {
                        if(c2[r]==b[0])
                        q[r]=p[i];
                        printf("\t%d",q[r]);
                }
                printf("\n");
        }
    }
}
printf("\nThe no of page faults is %d",c);
}
```

OUTPUT:

```
Enter no of pages:10
Enter the reference string:7 5 9 4 3 7 9 6 2 1
Enter no of frames:3
        7
        7       5
        7       5       9
        4       5       9
        4       3       9
        4       3       7
        9       3       7
        9       6       7
        9       6       2
        1       6       2

The no of page faults is 10
```